Plan

Two lectures on the OUU paper:

- Value-based uncertainty and multi-objective optimization
- Understanding orders and Functorial uncertainty

Plan

Two lectures on the OUU paper:

- Value-based uncertainty and multi-objective optimization
- Understanding orders and Functorial uncertainty

Today:

- Understanding orders and Functorial uncertainty

The foundation: axioms of relations

- Before we optimize, we must know how to compare.
  A binary relation in Haskell is simply a function:

$$\mathit{type}\ \mathit{Rel}\ a\ =\ a\ \rightarrow\ a\ \rightarrow\ \mathit{Bool}$$

The foundation: axioms of relations

- Before we optimize, we must know how to compare.
  A binary relation in Haskell is simply a function:

$$\mathit{type}\ \mathit{Rel}\ a\ =\ a\ \rightarrow\ a\ \rightarrow\ \mathit{Bool}$$

- We classify a relation R (like $\leqslant$) based on the mathematical axioms it satisfies:
  - Reflexivity: Everything relates to itself.
    $\mathit{reflexive}\ (\leqslant)\ x\ =\ x\ \leqslant\ x$
  - Transitivity: Chains collapse.
    $\mathit{transitive}\ (\leqslant)\ x\ y\ z\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ z)\ \Rightarrow\ (x\ \leqslant\ z)$
  - Anti-symmetry: Mutual relation implies equality.
    $\mathit{antiSymmetric}\ (\leqslant)\ x\ y\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ x)\ \Rightarrow\ (x\ ==\ y)$
  - Totality: Any two elements can be compared.
    $\mathit{total}\ (\leqslant)\ x\ y\ =\ x\ \leqslant\ y\ \vee\ y\ \leqslant\ x$

The foundation: axioms of relations

- Before we optimize, we must know how to compare.
  A binary relation in Haskell is simply a function:

$$\mathit{type}\ \mathit{Rel}\ a\ =\ a\ \rightarrow\ a\ \rightarrow\ \mathit{Bool}$$

- We classify a relation R (like $\leqslant$) based on the mathematical axioms it satisfies:
  - Reflexivity: Everything relates to itself.
    $\mathit{reflexive}\ (\leqslant)\ x\ =\ x\ \leqslant\ x$
  - Transitivity: Chains collapse.
    $\mathit{transitive}\ (\leqslant)\ x\ y\ z\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ z)\ \Rightarrow\ (x\ \leqslant\ z)$
  - Anti-symmetry: Mutual relation implies equality.
    $\mathit{antiSymmetric}\ (\leqslant)\ x\ y\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ x)\ \Rightarrow\ (x\ ==\ y)$
  - Totality: Any two elements can be compared.
    $\mathit{total}\ (\leqslant)\ x\ y\ =\ x\ \leqslant\ y\ \vee\ y\ \leqslant\ x$

The foundation: axioms of relations

- Before we optimize, we must know how to compare.
  A binary relation in Haskell is simply a function:

$$\mathit{type}\ \mathit{Rel}\ a\ =\ a\ \rightarrow\ a\ \rightarrow\ \mathit{Bool}$$

- We classify a relation R (like $\leqslant$) based on the mathematical axioms it satisfies:
  - Reflexivity: Everything relates to itself.
    $\mathit{reflexive}\ (\leqslant)\ x\ =\ x\ \leqslant\ x$
  - Transitivity: Chains collapse.
    $\mathit{transitive}\ (\leqslant)\ x\ y\ z\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ z)\ \Rightarrow\ (x\ \leqslant\ z)$
  - Anti-symmetry: Mutual relation implies equality.
    $\mathit{antiSymmetric}\ (\leqslant)\ x\ y\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ x)\ \Rightarrow\ (x\ ==\ y)$
  - Totality: Any two elements can be compared.
    $\mathit{total}\ (\leqslant)\ x\ y\ =\ x\ \leqslant\ y\ \vee\ y\ \leqslant\ x$

The foundation: axioms of relations

- Before we optimize, we must know how to compare.
  A binary relation in Haskell is simply a function:

$$\mathit{type}\ \mathit{Rel}\ a\ =\ a\ \rightarrow\ a\ \rightarrow\ \mathit{Bool}$$

- We classify a relation R (like $\leqslant$) based on the mathematical axioms it satisfies:
  - Reflexivity: Everything relates to itself.
    $\mathit{reflexive}\ (\leqslant)\ x\ =\ x\ \leqslant\ x$
  - Transitivity: Chains collapse.
    $\mathit{transitive}\ (\leqslant)\ x\ y\ z\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ z)\ \Rightarrow\ (x\ \leqslant\ z)$
  - Anti-symmetry: Mutual relation implies equality.
    $\mathit{antiSymmetric}\ (\leqslant)\ x\ y\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ x)\ \Rightarrow\ (x\ ==\ y)$
  - Totality: Any two elements can be compared.
    $\mathit{total}\ (\leqslant)\ x\ y\ =\ x\ \leqslant\ y\ \vee\ y\ \leqslant\ x$

The foundation: axioms of relations

- Before we optimize, we must know how to compare.
  A binary relation in Haskell is simply a function:

$$\mathit{type}\ \mathit{Rel}\ a\ =\ a\ \rightarrow\ a\ \rightarrow\ \mathit{Bool}$$

- We classify a relation R (like $\leqslant$) based on the mathematical axioms it satisfies:
  - Reflexivity: Everything relates to itself.
    $\mathit{reflexive}\ (\leqslant)\ x\ =\ x\ \leqslant\ x$
  - Transitivity: Chains collapse.
    $\mathit{transitive}\ (\leqslant)\ x\ y\ z\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ z)\ \Rightarrow\ (x\ \leqslant\ z)$
  - Anti-symmetry: Mutual relation implies equality.
    $\mathit{antiSymmetric}\ (\leqslant)\ x\ y\ =\ (x\ \leqslant\ y\ \wedge\ y\ \leqslant\ x)\ \Rightarrow\ (x\ \texttt{==}\ y)$
  - Totality: Any two elements can be compared.
    $\mathit{total}\ (\leqslant)\ x\ y\ =\ x\ \leqslant\ y\ \vee\ y\ \leqslant\ x$

The order hierarchy

- By combining these axioms, we build the hierarchy of orders used in optimization:

The order hierarchy

- By combining these axioms, we build the hierarchy of orders used in optimization:
- **Preorder (reflexive + transitive):** The bare minimum for optimization. Elements can be compared, and chains make sense, but distinct items can be mutually "better or equal" to each other (cycles are possible).

The order hierarchy

- By combining these axioms, we build the hierarchy of orders used in optimization:
- **Preorder (reflexive + transitive):** The bare minimum for optimization. Elements can be compared, and chains make sense, but distinct items can be mutually "better or equal" to each other (cycles are possible).
- **Partial order (preorder + anti-symmetry):** Prevents cycles. If $x \leqslant y$ and $y \leqslant x$, they are the exact same object ($x == y$). However, some pairs might be completely incomparable.

The order hierarchy

- By combining these axioms, we build the hierarchy of orders used in optimization:
- **Preorder (reflexive + transitive):** The bare minimum for optimization. Elements can be compared, and chains make sense, but distinct items can be mutually "better or equal" to each other (cycles are possible).
- **Partial order (preorder + anti-symmetry):** Prevents cycles. If $x \leqslant y$ and $y \leqslant x$, they are the exact same object ($x == y$). However, some pairs might be completely incomparable.
- **Total order (partial order + totality):** The standard $\leqslant$ for numbers. Every element can be compared to every other element. *This guarantees a single, unique minimum.*

Exercises: exploring the hierarchy

Exercise 2.1

Provide a concrete, simple example of a relation for each level in the hierarchy such that it resides *strictly* on that level.

1. A preorder that is *not* a partial order.
2. A partial order that is *not* a total order.
3. A total order.

Exercises: exploring the hierarchy

Exercise 2.1

Provide a concrete, simple example of a relation for each level in the hierarchy such that it resides *strictly* on that level.

1. A preorder that is *not* a partial order.
2. A partial order that is *not* a total order.
3. A total order.

Exercise 2.2

Think about the extremes. What is the "minimal" valid preorder you can define on a set (the one with the fewest related pairs)? What is the "maximal" valid preorder (the one with the most related pairs)?

Building relations: the *on* combinator

- How do we build complex relations from simple ones?
  Haskell provides a beautiful tool in *Data.Function*:

$$on :: (b \to b \to c) \to (a \to b) \to a \to a \to c$$

Building relations: the *on* combinator

- How do we build complex relations from simple ones?
  Haskell provides a beautiful tool in *Data.Function*:

$$on :: (b \to b \to c) \to (a \to b) \to a \to a \to c$$

- It applies a function to both arguments before combining them.
  We can use it to lift a relation on B to a relation on A:

$$(\leqslant_A) = (\leqslant_B) \text{ `}on\text{` } f$$
$$\text{-- which means:}$$
$$x \leqslant_A y = f\ x \leqslant_B f\ y$$

Building relations: the *on* combinator

- How do we build complex relations from simple ones?
  Haskell provides a beautiful tool in *Data.Function*:

$$on :: (b \to b \to c) \to (a \to b) \to a \to a \to c$$

- It applies a function to both arguments before combining them.
  We can use it to lift a relation on B to a relation on A:

$$(\leqslant_A) = (\leqslant_B) \text{ `}on\text{` } f$$
$$\text{-- which means:}$$
$$x \leqslant_A y = f\ x \leqslant_B f\ y$$

- **Hierarchy impact:** If $\leqslant_B$ is a preorder, and f is any function, then $\leqslant_A$ is automatically a preorder. (However, anti-symmetry is usually lost unless f is injective).

Building relations: product orders

- In Multi-Objective Optimization (MOO), we evaluate multiple functions, resulting in tuples. How do we order pairs (A, B)?

Building relations: product orders

- In Multi-Objective Optimization (MOO), we evaluate multiple functions, resulting in tuples. How do we order pairs (A, B)?
- We define the product order by checking both components simultaneously:

$$(x_1 , y_1) \leqslant_{A \times B} (x_2 , y_2) = (x_1 \leqslant_A x_2) \wedge (y_1 \leqslant_B y_2)$$

Building relations: product orders

- In Multi-Objective Optimization (MOO), we evaluate multiple functions, resulting in tuples. How do we order pairs (A, B)?
- We define the product order by checking both components simultaneously:

$$(x_1 , y_1) \leqslant_{A \times B} (x_2 , y_2) = (x_1 \leqslant_A x_2) \wedge (y_1 \leqslant_B y_2)$$

- **What happens to the hierarchy?**
  - reflexivity? *preserved.*
  - transitivity? *preserved.*
  - anti-symmetry? *preserved.*
  - totality? **lost!**

Building relations: product orders

- In Multi-Objective Optimization (MOO), we evaluate multiple functions, resulting in tuples. How do we order pairs (A, B)?
- We define the product order by checking both components simultaneously:

$$(x_1 , y_1) \leqslant_{A \times B} (x_2 , y_2) = (x_1 \leqslant_A x_2) \wedge (y_1 \leqslant_B y_2)$$

- **What happens to the hierarchy?**
  - reflexivity? *preserved.*
  - transitivity? *preserved.*
  - anti-symmetry? *preserved.*
  - totality? **lost!**

Building relations: product orders

- In Multi-Objective Optimization (MOO), we evaluate multiple functions, resulting in tuples. How do we order pairs (A, B)?
- We define the product order by checking both components simultaneously:

$$(x_1 , y_1) \leqslant_{A \times B} (x_2 , y_2) = (x_1 \leqslant_A x_2) \wedge (y_1 \leqslant_B y_2)$$

- **What happens to the hierarchy?**
  - reflexivity? *preserved.*
  - transitivity? *preserved.*
  - anti-symmetry? *preserved.*
  - totality? **lost!**
- Even if $\leqslant_A$ and $\leqslant_B$ are total orders, their product is strictly a **partial** order. (E.g., $(1 , 2)$ and $(2 , 1)$ are incomparable). This loss of totality is the birthplace of the Pareto front!

Exercises: product orders

Exercise 2.3

Assume we have two domains: *String* ordered by length (a preorder), and *Int* ordered by standard $\leqslant$ (a total order).

1. Define the product order $\leqslant_{A \times B}$ for the tuple (*String*, *Int*).
2. Provide an example of two tuples that are strictly ordered (one dominates the other).
3. Provide an example of two tuples that are mutually incomparable under this product order.

Functorial uncertainty: beyond single values

- In many decision problems, outcomes are not precisely known.
  We distinguish between two types of functorial uncertainty:
  - epistemic uncertainty: parameters or initial conditions are unknown.
  - aleatoric uncertainty: the system itself is non-deterministic (e.g., stochastic processes).

Functorial uncertainty: beyond single values

- In many decision problems, outcomes are not precisely known.
  We distinguish between two types of functorial uncertainty:
  - epistemic uncertainty: parameters or initial conditions are unknown.
  - aleatoric uncertainty: the system itself is non-deterministic (e.g., stochastic processes).

Functorial uncertainty: beyond single values

- In many decision problems, outcomes are not precisely known.
  We distinguish between two types of functorial uncertainty:
  - epistemic uncertainty: parameters or initial conditions are unknown.
  - aleatoric uncertainty: the system itself is non-deterministic (e.g., stochastic processes).

Functorial uncertainty: beyond single values

- In many decision problems, outcomes are not precisely known.
  We distinguish between two types of functorial uncertainty:
  - epistemic uncertainty: parameters or initial conditions are unknown.
  - aleatoric uncertainty: the system itself is non-deterministic (e.g., stochastic processes).
- Instead of $f : A \to B$, we model these problems with functions returning values in an uncertainty functor U:

$$f : A \to U\,B$$

Functorial uncertainty: beyond single values

- In many decision problems, outcomes are not precisely known.
  We distinguish between two types of functorial uncertainty:
  - epistemic uncertainty: parameters or initial conditions are unknown.
  - aleatoric uncertainty: the system itself is non-deterministic (e.g., stochastic processes).
- Instead of $f : A \to B$, we model these problems with functions returning values in an uncertainty functor U:

$$f : A \to U\,B$$

- Examples of U include *Identity*, *Maybe*, *Set*, *List*, or probability distributions.

Functorial uncertainty: beyond single values

- In many decision problems, outcomes are not precisely known.
  We distinguish between two types of functorial uncertainty:
  - epistemic uncertainty: parameters or initial conditions are unknown.
  - aleatoric uncertainty: the system itself is non-deterministic (e.g., stochastic processes).
- Instead of $f : A \to B$, we model these problems with functions returning values in an uncertainty functor U:

$$f : A \to U\,B$$

- Examples of U include *Identity*, *Maybe*, *Set*, *List*, or probability distributions.

Exercise 2.4

In the context of climate policy, give one concrete example of epistemic uncertainty and one concrete example of aleatoric uncertainty.

Minimization under functorial uncertainty

- To minimize $f : A \to U\ B$, we cannot rely on a simple total order on B. We introduce a measure function $\mu : U\ B \to C$ (where C is totally ordered).

Minimization under functorial uncertainty

- To minimize $f : A \to U\ B$, we cannot rely on a simple total order on B. We introduce a measure function $\mu : U\ B \to C$ (where C is totally ordered).
- Using the *on* combinator we just learned, this builds a new relation $\leqslant_\mu$ on $U\ B$:

$$(\leqslant_\mu) = (\leqslant_C)\ `on`\ \mu$$

Minimization under functorial uncertainty

- To minimize $f : A \to U\ B$, we cannot rely on a simple total order on B. We introduce a measure function $\mu : U\ B \to C$ (where C is totally ordered).
- Using the *on* combinator we just learned, this builds a new relation $\leqslant_\mu$ on $U\ B$:

$$(\leqslant_\mu) = (\leqslant_C)\ `on`\ \mu$$

- We generalize *minOf* and *argminOf* to take this measure function into account:

$$\begin{aligned} &minOfU &&:: (Ord\ b, Ord\ c) \Rightarrow (u\ b \to c) \to (a \to u\ b) \to Set\ a \to Set\ (u\ b) \\ &argminOfU &&:: (Ord\ b, Ord\ c, Eq\ (u\ b)) \Rightarrow (u\ b \to c) \to (a \to u\ b) \to Set\ a \to Set\ a \end{aligned}$$

- Using a measure function like "expected value" is common but implicitly assumes *risk neutrality*. Our formulation allows for any valid measure (e.g., worst-case for risk aversion).

Minimization under functorial uncertainty

- To minimize $f : A \to U\ B$, we cannot rely on a simple total order on B. We introduce a measure function $\mu : U\ B \to C$ (where C is totally ordered).
- Using the *on* combinator we just learned, this builds a new relation $\leqslant_\mu$ on $U\ B$:

$$(\leqslant_\mu) = (\leqslant_C)\ `on`\ \mu$$

- We generalize *minOf* and *argminOf* to take this measure function into account:

$$\begin{aligned} &minOfU &&:: (Ord\ b, Ord\ c) \Rightarrow (u\ b \to c) \to (a \to u\ b) \to Set\ a \to Set\ (u\ b) \\ &argminOfU &&:: (Ord\ b, Ord\ c, Eq\ (u\ b)) \Rightarrow (u\ b \to c) \to (a \to u\ b) \to Set\ a \to Set\ a \end{aligned}$$

- Using a measure function like "expected value" is common but implicitly assumes *risk neutrality*. Our formulation allows for any valid measure (e.g., worst-case for risk aversion).

Exercise 2.5

Compare the type signatures of *argminOf* (from Section 1) and *argminOfU* above. Why is the additional class constraint $Eq\ (u\ b)$ required to implement *argminOfU*?

Structural comparison of uncertain values

- To define soundness, we need a way to say one uncertain outcome is strictly better than another, *regardless* of the chosen measure.

Structural comparison of uncertain values

- To define soundness, we need a way to say one uncertain outcome is strictly better than another, *regardless* of the chosen measure.
- We define a relation ($\prec_u$) meaning "universally strictly better":

$$ub_1 \prec_u ub_2 \Leftrightarrow \forall\, x \in ub_1.\ \forall\, y \in ub_2.\ x < y$$

Structural comparison of uncertain values

- To define soundness, we need a way to say one uncertain outcome is strictly better than another, *regardless* of the chosen measure.
- We define a relation ($\prec_u$) meaning "universally strictly better":

$$ub_1 \prec_u ub_2 \Leftrightarrow \forall\, x \in ub_1.\ \forall\, y \in ub_2.\ x < y$$

- We also derive its negation, meaning "not structurally strictly dominated by":

$$ub_1 \not\prec_u ub_2 \Leftrightarrow \neg(ub_2 \prec_u ub_1)$$

Structural comparison of uncertain values

- To define soundness, we need a way to say one uncertain outcome is strictly better than another, *regardless* of the chosen measure.
- We define a relation ($\prec_u$) meaning "universally strictly better":

$$ub_1 \prec_u ub_2 \Leftrightarrow \forall\, x \in ub_1.\ \forall\, y \in ub_2.\ x < y$$

- We also derive its negation, meaning "not structurally strictly dominated by":

$$ub_1 \not\prec_u ub_2 \Leftrightarrow \neg(ub_2 \prec_u ub_1)$$

Exercise 2.6

Assume $U = List$ and $B = Int$. Evaluate these examples and discuss properties of ($\prec_u$):

1. $[1, 2\ ] \prec_u [3, 4\ ]$
2. $[1, 3\ ] \prec_u [2, 4\ ]$
3. $[3, 2\ , 2\ ] \prec_u [1, 1\ , 0\ ]$

Soundness specifications for *minOfU*

- A sound minimization strategy should *never* return a result that is structurally strictly dominated by another available option.

Soundness specifications for *minOfU*

- A sound minimization strategy should *never* return a result that is structurally strictly dominated by another available option.

- This gives us our core consistency specification for *minOfU*:

  1. $\forall\ a \in as.\ \forall\ ub \in minOfU\ \mu\ f\ as.\ ub \precsim_u f\ a$
  2. $\forall\ a \in as.\ (a \in argminOfU\ \mu\ f\ as) \Rightarrow (f\ a \in minOfU\ \mu\ f\ as)$
  3. $\forall\ a \in as.\ (f\ a \in minOfU\ \mu\ f\ as) \Rightarrow (a \in argminOfU\ \mu\ f\ as)$

Soundness specifications for *minOfU*

- A sound minimization strategy should *never* return a result that is structurally strictly dominated by another available option.

- This gives us our core consistency specification for *minOfU*:

  1. $\forall\ a \in as.\ \forall\ ub \in minOfU\ \mu\ f\ as.\ ub \precsim_u f\ a$
  2. $\forall\ a \in as.\ (a \in argminOfU\ \mu\ f\ as) \Rightarrow (f\ a \in minOfU\ \mu\ f\ as)$
  3. $\forall\ a \in as.\ (f\ a \in minOfU\ \mu\ f\ as) \Rightarrow (a \in argminOfU\ \mu\ f\ as)$

Exercise 2.7

Why does the specification rule (1) use $\precsim_u$ instead of a standard $\leqslant$?

Monotonicity conditions for measures

- Which measure functions (μ) guarantee that *minOfU* satisfies the soundness specifications?

Monotonicity conditions for measures

- Which measure functions (μ) guarantee that *minOfU* satisfies the soundness specifications?
- **Pointwise monotonicity (M1):** If we increase elements without changing the structure's shape ($ub_1 \leqslant\cdot\ ub_2$), the measure should not decrease.

$$ub_1 \leqslant\cdot\ ub_2 \implies \mu\ ub_1 \leqslant \mu\ ub_2$$

Monotonicity conditions for measures

- Which measure functions (μ) guarantee that *minOfU* satisfies the soundness specifications?
- **Pointwise monotonicity (M1):** If we increase elements without changing the structure's shape ($ub_1 \leqslant\cdot\ ub_2$), the measure should not decrease.

$$ub_1 \leqslant\cdot\ ub_2 \implies \mu\ ub_1 \leqslant \mu\ ub_2$$

- **Structural monotonicity (M2):** If a structure is universally strictly better ($ub_1 \prec_u ub_2$), the measure *must* be strictly lower.

$$ub_1 \prec_u ub_2 \implies \mu\ ub_1 < \mu\ ub_2$$

Monotonicity conditions for measures

- Which measure functions (μ) guarantee that *minOfU* satisfies the soundness specifications?
- **Pointwise monotonicity (M1):** If we increase elements without changing the structure's shape ($ub_1 \leqslant\cdot\ ub_2$), the measure should not decrease.

$$ub_1 \leqslant\cdot\ ub_2 \Rightarrow \mu\ ub_1 \leqslant \mu\ ub_2$$

- **Structural monotonicity (M2):** If a structure is universally strictly better ($ub_1 \prec_u ub_2$), the measure *must* be strictly lower.

$$ub_1 \prec_u ub_2 \Rightarrow \mu\ ub_1 < \mu\ ub_2$$

Exercise 2.8

Consider $U = List$ and the measure function $\mu = const\ 42$. Does this measure satisfy (M1)? Does it satisfy (M2)? Explain how using this measure would cause *minOfU* to violate the soundness specification.

Lifting relations with ∀∀

- To compare uncertain outcomes, we lifted a base relation R on B to a relation R_u on $U\ B$. The strictest way is the ∀∀-lift:

$$ub_1\ `Ru`\ ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x\ `R`\ y$$

Lifting relations with ∀∀

- To compare uncertain outcomes, we lifted a base relation R on B to a relation R_u on $U\ B$. The strictest way is the ∀∀-lift:

$$ub_1\ `Ru`\ ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x\ `R`\ y$$

- In our code, we used this to define strict structural dominance ($\prec_u$) based on the underlying strict order $<$:

$$ub_1 \prec_u ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x < y$$

Lifting relations with ∀∀

- To compare uncertain outcomes, we lifted a base relation R on B to a relation R_u on $U\ B$. The strictest way is the ∀∀-lift:

$$ub_1\ `Ru`\ ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x\ `R`\ y$$

- In our code, we used this to define strict structural dominance ($\prec_u$) based on the underlying strict order $<$:

$$ub_1 \prec_u ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x < y$$

- But what happens to the mathematical properties of the base relation when we lift it this way? Do they survive?

Lifting relations with ∀∀

- To compare uncertain outcomes, we lifted a base relation R on B to a relation R_u on $U\ B$. The strictest way is the ∀∀-lift:

$$ub_1\ `Ru`\ ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x\ `R`\ y$$

- In our code, we used this to define strict structural dominance ($\prec_u$) based on the underlying strict order $<$:

$$ub_1 \prec_u ub_2 \Leftrightarrow \forall\ x \in ub_1.\ \forall\ y \in ub_2.\ x < y$$

- But what happens to the mathematical properties of the base relation when we lift it this way? Do they survive?

Exercise 2.9

Assume the base relation $<$ is *irreflexive* ($\forall\ x.\ \neg(x < x)$). Prove that the lifted relation ($\prec_u$) is also irreflexive for any non-empty set ub. What happens if ub is the empty set?

Transitivity and the empty set trap

- A strict dominance relation must be transitive to be useful for optimization. Let's check the transitivity of $(\prec_u)$.

Transitivity and the empty set trap

- A strict dominance relation must be transitive to be useful for optimization. Let's check the transitivity of $(\prec_u)$.
- Assume the base relation $<$ is transitive. By definition:
  - $\forall\ x\ \in\ ub_1.\ \ \forall\ y\ \in\ ub_2.\ \ x < y$
  - $\forall\ y\ \in\ ub_2.\ \ \forall\ z\ \in\ ub_3.\ \ y < z$

Transitivity and the empty set trap

- A strict dominance relation must be transitive to be useful for optimization. Let's check the transitivity of $(\prec_u)$.
- Assume the base relation $<$ is transitive. By definition:
  - $\forall\ x\ \in\ ub_1.\ \ \forall\ y\ \in\ ub_2.\ \ x < y$
  - $\forall\ y\ \in\ ub_2.\ \ \forall\ z\ \in\ ub_3.\ \ y < z$
- We want to conclude $\forall\ x\ \in\ ub_1.\ \ \forall\ z\ \in\ ub_3.\ \ x < z$. We can use $x < y$ and $y < z$ to get $x < z$... *but only if a y actually exists!*

Transitivity and the empty set trap

- A strict dominance relation must be transitive to be useful for optimization. Let's check the transitivity of $(\prec_u)$.
- Assume the base relation $<$ is transitive. By definition:
  - $\forall\ x\ \in\ ub_1.\ \ \forall\ y\ \in\ ub_2.\ \ x < y$
  - $\forall\ y\ \in\ ub_2.\ \ \forall\ z\ \in\ ub_3.\ \ y < z$
- We want to conclude $\forall\ x\ \in\ ub_1.\ \ \forall\ z\ \in\ ub_3.\ \ x < z$. We can use $x < y$ and $y < z$ to get $x < z$... *but only if a y actually exists!*

Exercise 2.10

Explain why the paper's Haskell test for the transitivity of $(\prec_u)$ explicitly includes the precondition $\neg(\mathit{null}\ \mathit{ys})$. What breaks if ys (the intermediate set ub_2) is empty?

The loss of reflexivity (why we can't $\forall\forall$-lift $\leqslant$)

- We successfully lifted the strict order $<$ to $(\prec_u)$. What if we try to lift the non-strict preorder $\leqslant$ using the same $\forall\forall$ logic?

The loss of reflexivity (why we can't ∀∀-lift $\leqslant$)

- We successfully lifted the strict order $<$ to $(\prec_u)$. What if we try to lift the non-strict preorder $\leqslant$ using the same ∀∀ logic?
- Let's define a hypothetical relation $\sqsubseteq_u$:

$$ub_1 \sqsubseteq_u ub_2 \Leftrightarrow \forall\, x \in ub_1.\ \forall\, y \in ub_2.\ x \leqslant y$$

The loss of reflexivity (why we can't $\forall\forall$-lift $\leqslant$)

- We successfully lifted the strict order $<$ to $(\prec_u)$. What if we try to lift the non-strict preorder $\leqslant$ using the same $\forall\forall$ logic?
- Let's define a hypothetical relation $\sqsubseteq_u$:

$$ub_1 \sqsubseteq_u ub_2 \Leftrightarrow \forall\, x \in ub_1.\ \forall\, y \in ub_2.\ x \leqslant y$$

- For this to be a valid preorder on uncertain outcomes, it *must* be reflexive ($ub \sqsubseteq_u ub$). Let's test it on a simple list: $ub = [1, 2]$.

The loss of reflexivity (why we can't ∀∀-lift $\leqslant$)

- We successfully lifted the strict order $<$ to $(\prec_u)$. What if we try to lift the non-strict preorder $\leqslant$ using the same ∀∀ logic?
- Let's define a hypothetical relation $\sqsubseteq_u$:

$$ub_1 \sqsubseteq_u ub_2 \Leftrightarrow \forall\, x \in ub_1.\ \forall\, y \in ub_2.\ x \leqslant y$$

- For this to be a valid preorder on uncertain outcomes, it *must* be reflexive ($ub \sqsubseteq_u ub$). Let's test it on a simple list: $ub = [1, 2]$.

Exercise 2.11

Evaluate $[1, 2] \sqsubseteq_u [1, 2]$ using the hypothetical definition above. What does this reveal about lifting preorders using the ∀∀ approach?

Summary: hierarchy impact of the ∀∀-lift

- Lifting relations using ∀∀ is mathematically harsh. Let's summarize what happens to the order hierarchy:

Summary: hierarchy impact of the ∀∀-lift

- Lifting relations using ∀∀ is mathematically harsh. Let's summarize what happens to the order hierarchy:

- **If the base relation is a strict total order ($<$):**
  - Irreflexivity: *preserved* (for non-empty U-structures).
  - Transitivity: *preserved* (for non-empty intermediate U-structures).
  - Totality: **lost completely.** Most U-structures overlap and are incomparable.
  - **Result:** A strict partial order ($\prec_u$). Useful for defining dominance boundaries.

Summary: hierarchy impact of the ∀∀-lift

- Lifting relations using ∀∀ is mathematically harsh. Let's summarize what happens to the order hierarchy:

- **If the base relation is a strict total order ($<$):**
  - Irreflexivity: *preserved* (for non-empty U-structures).
  - Transitivity: *preserved* (for non-empty intermediate U-structures).
  - Totality: **lost completely.** Most U-structures overlap and are incomparable.
  - **Result:** A strict partial order ($\prec_u$). Useful for defining dominance boundaries.

- **If the base relation is a total preorder ($\leqslant$):**
  - Reflexivity: **lost completely** (for U-structures of > 1 element).
  - **Result:** Not even a preorder! It becomes mathematically useless for finding optima.

Alternative: pointwise lifting

- If $\forall\forall$ is too destructive for non-strict relations like $\leqslant$, how did we define $ub_1 \leqslant\cdot\ ub_2$ in our M1 Monotonicity condition?

Alternative: pointwise lifting

- If $\forall\forall$ is too destructive for non-strict relations like $\leqslant$, how did we define $ub_1 \leqslant\cdot\ ub_2$ in our M1 Monotonicity condition?
- We used pointwise lifting. Instead of comparing every element to every other element, we only compare elements at the *same structural index*:

$$ub_1 \leqslant\cdot\ ub_2 \ = \ and\ (zipWith\ (\leqslant)\ ub_1\ ub_2)$$

Alternative: pointwise lifting

- If $\forall\forall$ is too destructive for non-strict relations like $\leqslant$, how did we define $ub_1 \leqslant\cdot\ ub_2$ in our M1 Monotonicity condition?
- We used pointwise lifting. Instead of comparing every element to every other element, we only compare elements at the *same structural index*:

$$ub_1 \leqslant\cdot\ ub_2 \ = \ and\ (zipWith\ (\leqslant)\ ub_1\ ub_2)$$

- This requires ub_1 and ub_2 to have the exact same "shape" (e.g., lists of the same length).

Alternative: pointwise lifting

- If $\forall\forall$ is too destructive for non-strict relations like $\leqslant$, how did we define $ub_1 \leqslant\cdot\ ub_2$ in our M1 Monotonicity condition?
- We used pointwise lifting. Instead of comparing every element to every other element, we only compare elements at the *same structural index*:

$$ub_1 \leqslant\cdot\ ub_2 \ = \ and\ (zipWith\ (\leqslant)\ ub_1\ ub_2)$$

- This requires ub_1 and ub_2 to have the exact same "shape" (e.g., lists of the same length).

Exercise 2.12

What happens to the order hierarchy under pointwise lifting? If $\leqslant$ is a total order, is $\leqslant\cdot$ a total order? Provide an example using $[1, 2\ ]$ and $[2, 1\ ]$.

Examples of uncertainty functors: intervals

- A common way to model uncertainty in physics is via intervals:

$$\textbf{data}\ I\ b\ =\ I\ (b,\ b)$$

meaning all values between the bounds are possible.

Examples of uncertainty functors: intervals

- A common way to model uncertainty in physics is via intervals:

  $$\textbf{data}\ I\ b\ =\ I\ (b,\ b)$$

  meaning all values between the bounds are possible.
- Structural comparison is defined by the endpoints:

  $$I\ (x_s\ ,\ x_e)\ \prec_u\ I\ (x'_s\ ,\ x'_e)\ =\ [\ x_s\ ,\ x_e\ ]\ \prec_u\ [\ x'_s\ ,\ x'_e\ ]$$

Examples of uncertainty functors: intervals

- A common way to model uncertainty in physics is via intervals:

$$\textbf{data}\ I\ b\ =\ I\ (b,\ b)$$

  meaning all values between the bounds are possible.

- Structural comparison is defined by the endpoints:

$$I\ (x_s\ ,\ x_e)\ \prec_u\ I\ (x'_s\ ,\ x'_e)\ =\ [\ x_s\ ,\ x_e\ ]\ \prec_u\ [\ x'_s\ ,\ x'_e\ ]$$

- Measures like *sumI* (sum of endpoints), *bestI* (lower bound), and *worstI* (upper bound) all satisfy (M2).

Examples of uncertainty functors: intervals

- A common way to model uncertainty in physics is via intervals:

  $$\textbf{data}\ I\ b\ =\ I\ (b,\ b)$$

  meaning all values between the bounds are possible.
- Structural comparison is defined by the endpoints:

  $$I\ (x_s\ ,\ x_e)\ \prec_u\ I\ (x'_s\ ,\ x'_e)\ =\ [\ x_s\ ,\ x_e\ ]\ \prec_u\ [\ x'_s\ ,\ x'_e\ ]$$

- Measures like *sumI* (sum of endpoints), *bestI* (lower bound), and *worstI* (upper bound) all satisfy (M2).

Exercise 2.13

Consider the measure function $widthI\ i\ =\ end\ i - start\ i$ (which measures the "width" of the uncertainty). Show that *widthI* violates condition (M2) by providing two specific intervals, i_1 and i_2, such that $i_1\ \prec_u\ i_2$ but $\neg(\mu\ i_1 < \mu\ i_2)$.